MASTER PRD

MULTI-TENANT BARBER & SALON BOOKING, BUSINESS MANAGEMENT & GROWTH SaaS

Version 1.0 — Complete Product & Engineering Specification

---

1. PROJECT OVERVIEW

Build a production-grade, multi-tenant SaaS platform for barbers, salons, beauty parlours, grooming studios, spas, makeup studios, hair studios and other appointment-based businesses.

This product must NOT be treated as a simple appointment-booking application.

The actual product vision is:

> **A complete business operating system and customer-growth platform for salons and barbers.**

The platform should help businesses:

- Register their business
- Create branches
- Manage staff
- Manage services
- Create service combos
- Set service duration
- Set prices
- Set rush/peak pricing
- Set low-demand promotions
- Manage working hours
- Manage staff schedules
- Manage holidays
- Accept online bookings
- Accept online payments
- Manage cancellations
- Manage rescheduling
- Manage customers
- Send offers
- Send notifications
- Bring customers back

- Promote empty slots
- Get discovered by nearby users
- Get sponsored placement
- View analytics
- Manage multiple branches
- Increase revenue
- Improve customer retention

The SaaS owner should eventually earn revenue from:

- ₹1 per chargeable booking
- Sponsored listings
- Promoted shops
- Future premium modules
- Future subscriptions
- Advanced marketing/analytics features

However, the entire platform must initially operate as a **free SaaS**.

---

2. CORE TECHNOLOGY STACK

Backend

Use:

- PHP 8.4+
- Laravel
- REST API
- Laravel Queue
- Laravel Scheduler
- Laravel Cache
- Laravel Sanctum or an appropriately secure token/session architecture
- MySQL

Mobile

Use:

- Flutter
- Dart
- Android first
- iOS architecture must remain possible later

Customer Web

Use a modern web frontend consuming the same REST API.

The exact frontend technology can be selected based on the existing workspace and environment, but do not duplicate backend/business logic.

Database

Primary:

- MySQL

Local mobile storage:

- SQLite where useful

Caching:

- Redis where available
- Laravel cache fallback where Redis is unavailable

Notifications

- Firebase Cloud Messaging
- In-app notification system

Payments

Create an abstraction layer supporting online payment gateways.

Potential providers:

- Razorpay
- PhonePe
- Cashfree
- Other compatible providers

Do not hard-code the application around one provider.

---

3. DEVELOPMENT AGENT RESPONSIBILITY

Antigravity will have access to the project folder.

It must behave as the primary development agent.

It must:

1. Inspect the entire workspace.
2. Identify existing projects.
3. Identify existing source code.
4. Identify existing Laravel/PHP/Flutter installations.
5. Identify package managers.
6. Identify database configuration.
7. Identify reusable components.
8. Preserve useful existing code.
9. Create missing directories.
10. Install required packages.
11. Configure the environment.
12. Build the backend.
13. Build the database.
14. Build APIs.
15. Build Flutter applications.
16. Build web applications.
17. Build admin panels.
18. Configure FCM.
19. Configure queues.
20. Configure caching.
21. Configure automated cleanup.
22. Configure application update infrastructure.
23. Implement testing.
24. Run tests.
25. Fix errors.
26. Optimize performance.
27. Perform security checks.
28. Build release APKs.
29. Document the entire system.

Do not merely generate a plan.

The goal is to actually implement the application.

---

4. EXISTING PROJECT SAFETY

Before modifying an existing workspace:

- Inspect it.
- Identify important files.
- Determine current architecture.

- Preserve reusable components.
- Do not blindly overwrite files.
- Do not delete existing functionality without verifying its purpose.
- Create backups before destructive migrations.

If an existing application is incomplete, improve it instead of unnecessarily rebuilding everything from zero.

---

5. PRODUCT ARCHITECTURE

The platform consists of:

```
                  SaaS Platform
                       |
            -------------------------
            |          |           |
         Backend     Database    Cache
         Laravel      MySQL      Redis
            |
    ------------------------
    |         |           |
Customer   Owner/Staff  Admin
Flutter     Flutter     Web
    |
Customer Web
```

All clients communicate with the same Laravel REST API.

The backend is the authoritative source of truth.

---

6. MULTI-TENANT ARCHITECTURE

Every business is a tenant.

Example:

```
Tenant A
├── Branch 1
├── Branch 2
├── Staff
├── Services
├── Customers
```

```
└── Bookings

Tenant B
├── Branch 1
├── Staff
├── Services
└── Bookings
```

Tenant A must NEVER access Tenant B's data.

Every tenant-specific API request must validate:

- authenticated user
- role
- tenant ownership
- branch access
- resource ownership

Never trust client-provided:

```
tenant_id
owner_id
branch_id
user_id
```

without backend validation.

---

7. USER ROLES

The platform must support:

```
SUPER_ADMIN
SUBADMIN
OWNER
STAFF
CUSTOMER
```

Implement granular permissions.

Example permissions:

```
view_dashboard
manage_branches
manage_services
manage_staff
manage_bookings
manage_customers
manage_promotions
view_reports
manage_payments
manage_billing
manage_notifications
manage_system
manage_app_versions
```

Frontend visibility is NOT security.

Backend authorization is mandatory.

---

8. SUPER ADMIN

The Super Admin is the SaaS owner.

Super Admin controls the entire platform.

Dashboard should show:

- Total businesses
- Active businesses
- Suspended businesses
- Total branches
- Total customers
- Total staff
- Total bookings
- Today's bookings
- Monthly bookings
- Chargeable bookings
- SaaS revenue
- Outstanding invoices
- Failed payments
- Active campaigns
- Notification statistics
- System health

Super Admin can:

- Manage tenants
- Verify tenants
- Suspend tenants
- Restore tenants
- Manage subadmins
- Manage permissions
- Manage billing
- Manage advertisements
- Manage sponsored shops
- Manage notifications
- Manage feature flags
- Manage remote configuration
- Manage app versions
- Manage platform settings
- View audit logs
- View reports
- Manage campaigns

---

9. SUBADMIN

Subadmins are employees of the SaaS company.

Each subadmin receives only the permissions assigned to them.

Example:

```
Support Subadmin
- View tenants
- View bookings
- Handle support

Billing Subadmin
- View invoices
- View billing
- Manage billing support

Marketing Subadmin
- Manage campaigns
- Manage advertisements
```

Never give all subadmins Super Admin access.

---

10. OWNER ACCOUNT

A business owner can register their business.

Registration information:

- Owner name
- Mobile number
- Email
- Password
- Business name
- Business type
- Logo
- Cover image
- Description
- Address
- City
- State
- PIN
- GPS coordinates
- Contact details

Business categories:

- Barber
- Salon
- Unisex Salon
- Beauty Parlour
- Grooming Studio
- Hair Studio
- Spa
- Makeup Studio
- Nail Studio
- Other

Architecture must support additional categories later.

---

11. OWNER VERIFICATION

Tenant status:

```
PENDING
UNDER_REVIEW
VERIFIED
```

```
REJECTED
SUSPENDED
```

Super Admin can verify businesses.

Public visibility can be restricted to verified businesses.

---

12. MULTI-BRANCH SUPPORT

An owner may have multiple outlets.

Example:

```
Owner
├── Kolkata Branch
├── Salt Lake Branch
├── New Town Branch
└── Howrah Branch
```

Each branch has its own:

- Address
- GPS location
- Working hours
- Holidays
- Staff
- Services
- Pricing
- Promotions
- Bookings
- Reports
- Availability
- Payment configuration

Owner can switch:

```
Current Branch
```

or select:

```
All Branches
```

for consolidated reporting.

---

13. OWNER AS STAFF

Owner can choose:

```
Can customers book the owner?
YES / NO
```

If YES:

- Owner becomes bookable
- Owner can be assigned services
- Owner has working hours
- Owner appears in staff selection

If NO:

- Owner remains management-only.

---

14. STAFF MANAGEMENT

Owner can add staff.

Initial standard configuration:

```
Owner + 3 staff
```

But staff limits must be configurable by the SaaS.

Staff profile:

- Name
- Photo
- Mobile
- Email
- Branch

- Services
- Working hours
- Breaks
- Leave
- Bookable status
- Active/inactive
- Role

---

15. STAFF-SERVICE MAPPING

Every staff member can have different capabilities.

Example:

```
Rahul
Hair Cut
Beard Trim

Amit
Hair Cut
Facial

Priya
Facial
Hair Styling
Makeup
```

Customer selecting Facial must only see eligible staff.

---

16. SERVICE MANAGEMENT

Owner can create services.

Fields:

- Service name
- Description
- Category
- Price
- Duration
- Buffer time
- Image

- Active/inactive
- Online booking enabled
- Branch
- Eligible staff
- Pricing rules

Example:

```
Hair Cut
₹150
30 minutes
```

---

17. SERVICE COMBOS

Owner can create packages.

Example:

```
Grooming Combo

Hair Cut
+
Beard Trim
+
Head Massage

Normal:
₹500

Combo:
₹399

Duration:
60 minutes
```

Combo fields:

- Name
- Included services
- Price
- Discount
- Duration
- Branch

- Staff eligibility
- Validity
- Active status

---

18. DYNAMIC PRICING

Owner can create time-based pricing.

Example:

```
10 AM – 1 PM
₹90

1 PM – 5 PM
₹70

5 PM – 9 PM
₹120
```

Support:

- Peak pricing
- Rush pricing
- Weekend pricing
- Holiday pricing
- Staff pricing
- Branch pricing
- Service pricing
- Promotional pricing
- Last-minute pricing
- Low-demand pricing

Never overwrite the original service price.

Store pricing rules separately.

---

19. LOW-DEMAND SLOT PROMOTIONS

If a business has empty slots, the owner can promote them.

Example:

```
2 PM – 4 PM
Normally ₹150

Special:
₹120
```

The system can display:

> Last-minute offer

to nearby customers.

Existing confirmed bookings must never have their price changed.

---

20. BUSINESS HOURS

Each branch has working hours.

Example:

```
Monday
10 AM – 9 PM

Tuesday
10 AM – 9 PM

Sunday
Closed
```

Support:

- Special hours
- Holidays
- Temporary closures
- Emergency closures

---

21. STAFF HOURS

Each staff member can have:

- Working days
- Working hours
- Breaks
- Leave
- Temporary unavailable status

The booking engine must use staff availability.

---

22. HOLIDAYS

Owner can define:

- Shop holidays
- Staff holidays
- Festival closures
- Special working days
- Emergency closures

---

23. REAL-TIME SHOP STATUS

System dynamically calculates:

```
OPEN
CLOSED
TEMPORARILY CLOSED
```

Example:

```
OPEN
Closes at 9:00 PM
```

or:

```
CLOSED
Opens tomorrow at 10:00 AM
```

---

24. CUSTOMER DISCOVERY

Customer installs the Flutter application.

After permission:

```
Current Location
        ↓
Nearby Businesses
        ↓
Distance
        ↓
Open/Closed
        ↓
Available slots
```

Show:

- Nearby
- Open Now
- Available Today
- Popular
- Top Rated
- Offers
- Sponsored

If location permission is denied, manual search must still work.

---

25. SEARCH

Search by:

- Shop name
- Service
- Staff
- Area
- City

- Category

Filters:

- Distance
- Price
- Open now
- Available today
- Rating
- Offers
- Sponsored

---

26. SHOP DETAIL PAGE

Display:

- Logo
- Cover
- Business name
- Rating
- Address
- Distance
- Open/closed
- Today's hours
- Services
- Combos
- Staff
- Offers
- Available slots
- Reviews
- Photos
- Directions
- Contact

Buttons:

```
BOOK NOW
FAVOURITE
SHARE
DIRECTIONS
```

---

27. BOOKING FLOW

Customer:

```
Select Shop
↓
Select Branch
↓
Select Service
↓
Select Combo if applicable
↓
Select Staff / Any Staff
↓
Select Date
↓
Select Time
↓
Review
↓
Online Payment
↓
Backend Verification
↓
Booking Confirmed
```

---

28. AVAILABILITY ENGINE

Availability is calculated on the backend.

Inputs:

- Branch
- Date
- Service
- Combo
- Staff
- Duration
- Buffer
- Working hours
- Breaks
- Holidays

- Leave
- Existing bookings
- Temporary closures
- Booking limits
- Advance booking rules

The frontend only displays backend results.

---

29. MULTIPLE SERVICES

Example:

```
Hair Cut = 30 min
Beard = 20 min
Facial = 45 min
```

The booking engine calculates total duration and staff compatibility.

---

30. ANY STAFF

Customer can choose:

```
Any Available Staff
```

Backend automatically selects an eligible staff member.

Selection should consider:

- Capability
- Availability
- Working hours
- Existing workload
- Fair allocation
- Owner configuration

---

31. SLOT LOCKING

Before payment:

```
AVAILABLE
↓
TEMPORARILY LOCKED
↓
PAYMENT
↓
CONFIRMED
```

Default lock:

```
5 minutes
```

Payment timeout:

```
LOCKED → AVAILABLE
```

Use:

- Transactions
- Row locking
- Unique constraints
- Expiration
- Idempotency

---

32. CONCURRENT BOOKING PROTECTION

If 100 customers try to book the same staff member/time:

The backend must determine the valid capacity.

Never allow duplicate successful booking.

This must be tested explicitly.

---

33. ONLINE PAYMENT

Current business requirement:

> ONLINE PAYMENT ONLY.

No cash option in the customer booking flow.

Payment must be abstracted through:

```
PaymentGatewayInterface
```

Possible implementations:

```
RazorpayGateway
PhonePeGateway
CashfreeGateway
```

---

34. PAYMENT VERIFICATION

Flutter cannot decide payment success.

Flow:

```
Payment
↓
Gateway
↓
Webhook / Verification
↓
Laravel
↓
Booking confirmation
```

Verify signatures/webhooks.

Payment webhooks must be idempotent.

---

35. CANCELLATION

Cancellation policy is controlled by the owner.

Example:

```
Free cancellation:
3 hours before appointment
```

Owner can choose the limit.

After the limit:

```
Cancellation unavailable
```

The booking must maintain cancellation history.

---

36. RESCHEDULING

Support rescheduling.

Process:

```
Request new time
↓
Check availability
↓
Lock new slot
↓
Release old slot
↓
Update booking
↓
Record history
↓
Notify customer/staff
```

---

37. WAITLIST

If no slot is available:

```
JOIN WAITLIST
```

Customer chooses:

- Date
- Time range
- Service
- Staff preference

When a slot opens:

```
FCM
+
In-app notification
```

The offer should expire after a configurable period.

---

38. WALK-IN ARCHITECTURE

Keep architecture ready for walk-in bookings.

Owner can create:

```
Walk-in customer
```

This can be expanded into a full POS/queue module later.

---

39. CUSTOMER BOOKING HISTORY

Customer can view:

```
Upcoming
Completed
Cancelled
Expired
```

Each booking:

- Shop
- Branch
- Service

- Staff
- Date
- Time
- Amount
- Payment status
- Booking status

Button:

```
BOOK AGAIN
```

---

40. CUSTOMER PROFILE

Fields:

- Name
- Mobile
- Email
- Photo
- Preferences
- Favourite shops
- Favourite staff
- History
- Notification settings
- Devices

Collect only information genuinely required.

---

41. FAVOURITES

Customers can favourite:

- Shops
- Staff
- Services

Display on home page.

---

42. REVIEWS

Only customers with completed bookings can review.

Support:

```
1–5 stars
Comment
```

Owner can respond.

Super Admin can moderate.

Prevent duplicate/fake reviews.

---

43. CUSTOMER CRM

Owner gets customer list.

Display:

- Name
- Contact
- Last booking
- Total bookings
- Total spend
- Favourite service
- Favourite staff

Segments:

```
New
Returning
VIP
Frequent
Inactive
```

---

44. CUSTOMER MARKETING

Owner can create campaigns:

- Offers
- Combos
- Discounts
- New services
- Festival promotions
- Last-minute slots
- Customer reactivation campaigns

Target permitted customer segments.

---

45. MARKETING CONSENT

Promotional notifications must respect customer preferences and consent.

Users can disable marketing notifications.

Transactional notifications remain separate.

---

46. NOTIFICATION SYSTEM

Use Firebase Cloud Messaging.

Notifications include:

Customer

- Booking confirmed
- Payment confirmed
- Booking reminder
- Rescheduled
- Cancelled
- Staff changed
- Shop closed
- Slot available
- Waitlist
- Offer
- Promotion

Owner

- New booking
- Payment received
- Cancellation
- Reschedule
- Staff change
- System notification
- Billing
- Platform announcement

Staff

- Assigned booking
- Schedule change
- Cancellation
- Reminder

---

47. IN-APP NOTIFICATION CENTER

Every application must have:

```
All
Bookings
Payments
Offers
System
```

Notifications contain:

```
id
user_id
type
title
body
data
read_at
created_at
```

Notifications should support deep links.

---

48. PROMOTIONAL NOTIFICATION FREQUENCY

The system must prevent spam.

Default:

```
Maximum 1 promotional campaign notification
per user
per 24 hours
```

unless explicitly configured otherwise.

Transactional/security notifications are not treated as promotional notifications.

---

49. PLATFORM ADVERTISEMENT SYSTEM

Super Admin can create:

- Banner ads
- Sponsored shops
- Promotional cards
- Platform announcements
- Featured businesses

Target by:

- All users
- City
- Region
- Nearby radius
- Category
- Customer segment

Ads must be clearly labelled.

---

50. SPONSORED SHOP SYSTEM

A shop can be featured:

```
SPONSORED
```

Potential locations:

- Home
- Nearby
- Search
- Category
- Offers

Sponsored placement should be clearly distinguished from organic ranking.

---

51. WATERMARK / BRANDING

Shop-specific pages can display:

- Shop logo
- Theme
- Brand colors
- Promotional watermark

The platform can also display:

```
Sponsored
```

or platform branding where applicable.

---

52. REMOTE UI SYSTEM

A major requirement is:

> Changes to themes, banners, page content and configurable UI should NOT require an APK update.

Create a remote configuration system.

The server controls:

- Theme
- Logo
- Colors

- Banners
- Home sections
- Feature flags
- Promotional cards
- Announcement cards
- Content
- Visibility
- Section order
- Shop promotion

---

53. REMOTE UI SAFETY

Do NOT download executable Flutter/Dart code.

Use predefined Flutter components.

Server sends structured configuration:

```
{
  "type": "banner",
  "title": "Weekend Offer",
  "image": "...",
  "action": "shop",
  "shop_id": 123
}
```

The application renders a known component.

This keeps remote customization flexible while maintaining security.

---

54. SERVER-CONTROLLED THEME

Remote theme:

```
primaryColor
secondaryColor
backgroundColor
surfaceColor
textColor
buttonStyle
```

```
borderRadius
logo
```

Shop-specific branding can be applied inside the shop experience without allowing tenant configuration to compromise the global application.

---

55. FEATURE FLAGS

Create:

```
feature_flags
```

Examples:

```
reviews
waitlist
promotions
sponsored
advanced_reports
loyalty
membership
billing
ai_insights
```

Features can be enabled/disabled remotely.

---

56. APPLICATION UPDATE SYSTEM

Because the platform will initially NOT be published through Google Play, the official website must distribute signed Android APKs.

Backend provides:

```
latest_version
minimum_version
download_url
release_notes
mandatory
```

```
checksum
release_date
```

The app checks periodically.

If a newer version exists:

```
New update available
Version 1.4.0

[Update Now]
```

If the installed version is below minimum:

```
Update Required
```

The app must use Android's normal secure installation/update process.

Never attempt silent unauthorized installation.

---

57. APK DISTRIBUTION

Official website:

```
/download
```

Provide:

```
Customer App
Owner App
Staff App
Latest Version
Release Notes
```

All APKs must be release-signed.

Protect signing keys.

Never distribute debug APKs.

58. CUSTOMER WEB

Customer can use web without installing the app.

Features:

- Registration
- Login
- Shop discovery
- Search
- Booking
- Payment
- Cancellation
- History
- Notifications
- Profile

Same backend and booking engine as Flutter.

---

59. OWNER/STAFF APPLICATION

Android Flutter application.

Screens:

```
Dashboard
Bookings
Calendar
Branches
Services
Combos
Staff
Customers
Promotions
Reports
Notifications
Payments
Billing
Settings
Profile
```

---

60. OWNER DASHBOARD

Show:

```
Today's bookings
Upcoming
Completed
Cancelled
Revenue
Customers
Staff availability
Branch status
Popular services
Peak hours
Promotions
```

---

61. STAFF DASHBOARD

Show only authorized information:

```
Today's bookings
Upcoming bookings
Current appointment
Schedule
Availability
Relevant customer information
Notifications
```

No owner-level financial/business settings.

---

62. MULTI-BRANCH REPORTING

Owner can see:

```
All Branches
```

or:

```
Branch A
Branch B
Branch C
```

Reports:

- Bookings
- Revenue
- Customers
- Services
- Staff
- Peak hours

---

63. BUSINESS ANALYTICS

Owner analytics:

Revenue

- Today
- Week
- Month
- Custom range

Bookings

- Total
- Completed
- Cancelled
- No-show
- Rescheduled

Customers

- New
- Returning
- Inactive
- VIP

Services

- Most booked
- Highest revenue

- Lowest demand

Staff

- Bookings
- Utilization
- Revenue contribution

Time

- Peak hours
- Low-demand hours

---

64. GROWTH ENGINE

The platform should actively help businesses increase income.

Examples:

```
Low-demand period detected.

Suggested:
20% discount from 2 PM – 4 PM.
```

Owner must approve such recommendations.

Never automatically change prices without explicit owner configuration.

---

65. CUSTOMER RETENTION

After service:

```
Thank you for visiting.

Book Again
```

After a configurable period:

```
It's been 30 days since your last visit.
Book your next appointment.
```

Respect marketing preferences.

---

66. REBOOKING

Customer can repeat previous booking.

System preselects:

- Shop
- Service
- Preferred staff

Then recalculates:

- Availability
- Current price
- Duration

before confirmation.

---

67. FUTURE LOYALTY SYSTEM

Architecture must support:

- Points
- Rewards
- Tiers
- Referral rewards

Not necessarily required for the first release.

---

68. FUTURE MEMBERSHIP

Architecture must support:

- Monthly membership
- Annual membership

- Discount membership
- Service packages
- Unlimited booking plans

---

69. FUTURE COUPONS

Architecture should support:

- Percentage discounts
- Fixed discounts
- Service-specific discounts
- Branch-specific discounts
- Customer-specific discounts
- Expiration

---

70. FUTURE GIFT CARDS

Architecture should allow:

- Gift cards
- Gift balances
- Recipient
- Expiry
- Redemption history

---

71. FUTURE REFERRAL SYSTEM

Customer can invite friends.

Potential:

```
Friend books
↓
Reward
```

Keep architecture ready.

---

72. FUTURE STAFF COMMISSION

Architecture should support:

```
Staff
Service
Commission %
Commission amount
```

---

73. FUTURE POS / INVENTORY

Keep the architecture extensible for:

- POS
- Product sales
- Inventory
- Stock
- Expenses

---

74. SAAS BILLING

Billing exists from day one but is disabled.

Initial:

```
billing_enabled = false
booking_fee_enabled = false
```

Future:

```
₹1 per chargeable booking
```

---

75. BILLING MUST BE CONFIGURABLE

Settings:

```
billing_enabled
booking_fee_enabled
booking_fee_amount
billing_currency
billing_start_date
billing_cycle
billing_due_days
grace_period
```

Do not hard-code ₹1 throughout the code.

---

76. CHARGEABLE BOOKING

Recommended chargeable state:

A booking becomes chargeable after reaching the configured valid state.

Do not charge for:

- Failed payments
- Expired payment sessions
- Duplicate requests
- Temporary slot locks
- Unconfirmed bookings
- Technical failures

The exact chargeable state must be configurable.

---

77. BILLING USAGE LEDGER

Create:

```
billing_usage
```

Fields:

```
tenant_id
booking_id
billing_cycle_id
amount
```

```
currency
status
created_at
```

One booking must create at most one usage record.

Use unique constraints/idempotency.

---

78. MONTHLY BILL

Example:

```
August 2026

742 chargeable bookings

742 × ₹1

Total:
₹742
```

Generate invoice.

---

79. BILLING ACTIVATION

Super Admin can activate billing:

```
Billing:
OFF

Booking fee:
₹1

Effective:
1 September 2026

[Enable]
```

Do not retroactively bill earlier free bookings unless explicitly configured.

---

80. BILLING ANNOUNCEMENT

When billing is activated, Super Admin can send:

> Important update: Starting from [date], the platform will charge ₹1 per chargeable booking. Your current free period remains available until [date]. Your usage bill will be generated at the end of the billing cycle.

Send:

- FCM
- In-app notification
- Admin banner

---

81. BILLING LIFECYCLE

```
FREE
↓
BILLING_ACTIVE
↓
INVOICE_GENERATED
↓
PAYMENT_DUE
↓
OVERDUE
↓
WARNING
↓
SUSPENDED
```

Successful payment:

```
SUSPENDED
↓
PAYMENT_SUCCESS
↓
ACTIVE
```

---

82. TENANT SUSPENSION

If bill remains unpaid after the configured grace period:

The tenant becomes suspended.

Suspended shop:

- Cannot accept new bookings
- Does not appear bookable
- Customers cannot create new bookings
- Owner can still log in
- Owner can see invoice
- Owner can pay outstanding amount
- Existing data remains intact

Never delete the tenant.

---

83. BILLING NOTIFICATIONS

Send:

- Billing activation
- Free period ending
- Invoice generated
- Payment due
- Reminder
- Final warning
- Suspension
- Successful payment
- Restoration

---

84. SAAS REVENUE SOURCES

Initial:

```
Free
```

Future:

```
₹1/booking
Sponsored shops
Featured placement
Premium analytics
Premium marketing
Membership module
Advanced CRM
Future subscription plans
```

Each monetization feature must be controlled by feature flags.

---

85. NOTIFICATION CAMPAIGN SYSTEM

Create campaign entities:

```
campaigns
campaign_audiences
campaign_deliveries
campaign_schedules
```

Campaign can be:

- Immediate
- Scheduled
- Expiring
- Location-based
- Segment-based

---

86. 24-HOUR PROMOTION SYSTEM

Users should not receive the same promotional campaign repeatedly within 24 hours.

Store delivery:

```
campaign_id
user_id
sent_at
```

Check before sending.

Default:

```
1 promotional notification / 24 hours / user
```

Transactional notifications are independent.

---

87. GEO-TARGETED MARKETING

Owner/platform can target:

```
5 km radius
```

or other configured radius.

Example:

> 20% OFF nearby.

Only users who have granted appropriate location access and enabled relevant promotional features should be targeted based on location.

---

88. CUSTOMER SEGMENTATION

Possible segments:

```
New Customer
Returning Customer
Inactive
Frequent
VIP
Recent booking
Specific service
Specific branch
```

Segmentation must be privacy-aware.

---

89. PLATFORM ADVERTISEMENTS

Support:

```
Banner
Card
Sponsored Shop
Promotion
Announcement
```

Each campaign has:

- Start
- End
- Audience
- Placement
- Priority
- Status

---

90. AD FREQUENCY

Do not overload users.

Configure:

```
Maximum banner frequency
Maximum popup frequency
Maximum promotional notification frequency
```

---

91. REMOTE CONTENT

Server can control:

- Home banners
- Promotional cards
- Shop recommendations
- Announcement text
- Help content
- Images

- Themes
- Feature visibility

without requiring an APK release.

---

92. SERVER-CONTROLLED PAGE STRUCTURE

Use safe predefined components.

Example:

```
{
  "sections": [
    {
      "type": "search"
    },
    {
      "type": "nearby_shops"
    },
    {
      "type": "sponsored_shop"
    },
    {
      "type": "offers"
    },
    {
      "type": "upcoming_booking"
    }
  ]
}
```

Flutter renders only known component types.

---

93. SELF-OPTIMIZATION

The application should maintain itself.

Flutter cleanup:

- Expired cache
- Temporary files
- Old cached images

- Obsolete notifications
- Stale local sessions

Backend cleanup:

- Expired slot locks
- Stale FCM tokens
- Expired waitlists
- Temporary files
- Temporary payment states
- Old cache
- Old logs according to retention policy

Never automatically delete:

- Financial records
- Required booking history
- Required audit records
- Required legal records

---

94. SQLITE

Use SQLite for useful local structured cache.

Possible:

- Recent shops
- Favourites
- Cached configuration
- Notification cache
- Recent booking information

SQLite is NOT authoritative.

Booking and payment must always be server verified.

---

95. REDIS

Use Redis when available for:

- Cache
- Locks
- Queues

- Rate limiting
- Temporary state

If Redis is unavailable, the application should gracefully use Laravel-supported fallback mechanisms where safe.

---

96. QUEUES

Use queues for:

- FCM
- Campaigns
- Emails
- Reports
- Invoice generation
- Reminders
- Waitlist notifications
- Billing
- Cleanup

Do not block critical booking requests unnecessarily.

---

97. SCHEDULER

Laravel Scheduler handles:

- Expired locks
- Reminders
- Billing
- Invoices
- Suspension
- Waitlists
- Cleanup
- Token cleanup
- Campaign scheduling

---

98. SECURITY ARCHITECTURE

Implement:

- HTTPS

- Secure authentication
- RBAC
- Tenant isolation
- Input validation
- SQL injection protection
- XSS protection
- CSRF where applicable
- CORS controls
- Rate limiting
- Secure password hashing
- Token security
- Refresh token rotation
- Secure local storage
- Payment webhook verification
- Audit logging
- Brute-force protection

---

99. DEVICE AUTHENTICATION

Do NOT use SIM-number detection as the authentication mechanism.

Use:

```
Account
+
Trusted device
+
Secure token
+
Optional biometric
+
Cryptographic device key
```

For high-security login approval:

```
Website login
↓
Server challenge
↓
FCM to registered phone
↓
User approves
↓
```

```
Phone signs challenge
↓
Server verifies
↓
Authentication succeeds
```

Private key never leaves the device.

---

100. PASSWORD SECURITY

Passwords:

- Must be hashed.
- Must never be logged.
- Must never be returned through APIs.
- Must never be stored in Flutter plaintext storage.

Use Laravel's secure hashing facilities.

---

101. API SECURITY

Every protected API must verify:

- Authentication
- Role
- Permission
- Tenant
- Resource ownership

Never trust:

```
user_id
tenant_id
branch_id
owner_id
price
booking status
payment status
```

from the client.

---

102. IDOR TESTING

Test:

```
User A requests User B's booking
```

Must fail.

Test:

```
Tenant A requests Tenant B's service
```

Must fail.

Test:

```
Staff requests Owner settings
```

Must fail.

---

103. RATE LIMITING

Rate-limit:

- Login
- Registration
- Password reset
- Booking
- Payment
- Notifications
- Admin actions
- Search where necessary

---

104. FILE UPLOAD SECURITY

Validate:

- MIME type

- Extension
- File size
- Image dimensions

Never trust user-provided file extensions.

Optimize uploaded images.

---

105. MONEY HANDLING

Use:

```
DECIMAL(12,2)
```

or integer minor units.

Never rely on floating-point calculations for financial accounting.

Store currency explicitly.

---

106. TIMEZONE

Database:

```
UTC
```

Tenant/branch:

```
timezone
```

India default:

```
Asia/Kolkata
```

Convert for display.

Do not depend on server timezone.

---

107. DATABASE STRUCTURE

Core tables should include:

```
users
roles
permissions
role_permissions
user_roles

tenants
tenant_settings
tenant_billing_profiles
tenant_subscriptions

branches
branch_settings
branch_hours
branch_holidays
branch_closures

staff
staff_branches
staff_services
staff_working_hours
staff_breaks
staff_leaves

service_categories
services
service_pricing_rules
service_combos
combo_services

customers
customer_devices
customer_preferences
customer_favourites

bookings
booking_items
booking_staff
booking_status_history
booking_slot_locks

payments
```

```
payment_transactions
payment_webhooks
refunds

notifications
notification_campaigns
notification_deliveries

promotions
promotion_rules
promotion_targets

reviews
waitlists

billing_cycles
billing_usage
invoices
invoice_items
billing_payments

feature_flags
remote_configs
app_versions

audit_logs
```

Add appropriate foreign keys, unique constraints and indexes.

---

108. API VERSIONING

Use:

```
/api/v1/
```

Future breaking changes can use:

```
/api/v2/
```

Never unexpectedly break old applications.

---

109. API RESPONSE FORMAT

Success:

```
{
  "success": true,
  "message": "Booking confirmed",
  "data": {}
}
```

Error:

```
{
  "success": false,
  "message": "Slot unavailable",
  "code": "SLOT_UNAVAILABLE"
}
```

Never expose stack traces in production.

---

110. IDEMPOTENCY

Use idempotency for:

- Booking
- Payment
- Payment webhook
- Refund
- Billing
- Notifications

Duplicate request must not create duplicate business effects.

---

111. PERFORMANCE

Optimize:

- SQL
- Indexes
- Query count

- N+1 queries
- Pagination
- Cache
- Queues
- Images
- API payloads

Use pagination.

Never return thousands of records unnecessarily.

---

112. DATABASE INDEXING

Important indexes include:

```
users.email
users.mobile

branches.tenant_id

services.branch_id
services.active

staff.branch_id
staff.active

bookings.branch_id
bookings.staff_id
bookings.start_at
bookings.end_at
bookings.status

payments.booking_id
payments.status

notifications.user_id
notifications.read_at

billing_usage.tenant_id
billing_usage.booking_id
billing_usage.billing_cycle_id
```

Add composite indexes based on actual query patterns.

---

113. GEOLOCATION

Nearby search must be handled efficiently.

Do not:

```
download every shop
↓
calculate distance in Flutter
```

Use database/geospatial querying or an appropriate indexed solution.

Return:

```
shop
distance
status
next available slot
```

with pagination.

---

114. OBSERVABILITY

Monitor:

- API errors
- Slow APIs
- Slow database queries
- Booking failures
- Payment failures
- Queue failures
- Notification failures
- Authentication failures

Do not log:

- Passwords
- Access tokens
- Refresh tokens
- Payment secrets
- Unnecessary personal information

---

115. BACKUP

Implement:

- Automated MySQL backups
- Retention policy
- Backup verification
- Restoration testing

A backup is not considered reliable until restoration has been tested.

---

116. DEVELOPMENT ENVIRONMENTS

Prepare:

```
Development
Staging
Production
```

Use separate:

- Environment variables
- Databases
- Credentials
- Storage
- API keys

Never commit production secrets.

---

117. ENVIRONMENT FILES

Generate:

```
.env.example
.env.required
```

Never commit real `.env`.

Document required variables.

---

118. FLUTTER ARCHITECTURE

Use feature-based architecture:

```
lib/
├── core/
│   ├── auth/
│   ├── config/
│   ├── network/
│   ├── notifications/
│   ├── routing/
│   ├── storage/
│   └── theme/
│
├── features/
│   ├── authentication/
│   ├── home/
│   ├── discovery/
│   ├── shops/
│   ├── services/
│   ├── booking/
│   ├── payment/
│   ├── history/
│   ├── favourites/
│   ├── notifications/
│   ├── promotions/
│   └── profile/
│
└── shared/
    ├── widgets/
    ├── models/
    └── utilities/
```

Do not put business logic inside UI widgets.

---

119. CUSTOMER APP

Customer application screens:

```
Splash
Onboarding
Login/Register
```

```
Home
Search
Nearby
Categories
Shop Details
Services
Staff
Booking
Payment
Booking Confirmation
Upcoming
History
Favourites
Offers
Notifications
Profile
Settings
```

---

120. OWNER APP

Owner application:

```
Login
Dashboard
Bookings
Calendar
Branches
Services
Combos
Staff
Customers
Promotions
Reports
Notifications
Payments
Billing
Settings
Profile
```

---

121. STAFF APP EXPERIENCE

Staff can access their restricted features through the same application if desired.

The application must determine the role from authenticated backend data.

Do not rely on a local role value for authorization.

---

122. CUSTOMER WEB APPLICATION

Use the same APIs.

Features:

- Search
- Discovery
- Booking
- Payment
- History
- Profile
- Notifications

---

123. SUPER ADMIN WEB PANEL

Sections:

```
Dashboard
Tenants
Branches
Customers
Staff
Bookings
Payments
Billing
Invoices
Campaigns
Advertisements
Notifications
Subadmins
Roles
Permissions
```

```
Feature Flags
Remote Config
App Versions
Audit Logs
System Settings
```

---

124. SUBADMIN PANEL

Display only permitted sections.

Direct API access must also be denied if permission is absent.

---

125. ONBOARDING

Owner setup:

```
Register
↓
Business
↓
Branch
↓
Services
↓
Staff
↓
Working hours
↓
Payment
↓
Booking policy
↓
Publish
```

Show setup progress.

---

126. CUSTOMER ONBOARDING

Customer:

```
Install
↓
Register/Login
↓
Location permission
↓
Explore
```

Do not force unnecessary forms before discovery.

---

127. DEEP LINKS

Support links for:

```
Shop
Booking
Offer
Campaign
Promotion
```

Notification taps should navigate correctly.

---

128. APP VERSION API

Create:

```
/api/v1/app/version
```

Response:

```
{
  "latest_version": "1.2.0",
  "minimum_version": "1.1.0",
  "download_url": "...",
  "release_notes": "...",
  "mandatory": false
}
```

---

129. APK UPDATE SECURITY

Use:

- HTTPS
- Official domain
- Release signing
- Checksum/integrity verification
- Version validation

Never put private signing keys into the repository.

---

130. REMOTE CONFIG CACHE

Flutter should cache remote configuration locally.

Process:

```
Launch
↓
Load cached configuration immediately
↓
Refresh configuration from server
↓
Apply latest valid configuration
```

This makes the application fast while still allowing server-controlled changes.

---

131. REMOTE CONFIG FAILURE

If server configuration is unavailable:

Use the last valid cached configuration.

If no cached configuration exists:

Use safe built-in defaults.

The app must not become unusable simply because remote configuration is temporarily unavailable.

---

132. SELF-HEALING CONFIGURATION

Validate remote configuration before applying.

If invalid:

```
Reject
↓
Use previous valid configuration
```

Do not crash the app.

---

133. AUTOMATED CLEANUP

Every cleanup job must:

- Have defined scope
- Have retention period
- Be logged
- Avoid critical records
- Be safe to run repeatedly

---

134. TESTING

Create:

Unit tests

- Pricing
- Availability
- Cancellation
- Billing
- Permissions
- Tenant isolation

Feature tests

- Registration
- Booking
- Payment

- Cancellation
- Rescheduling
- Staff
- Branches
- Promotions
- Billing

API tests

Every API endpoint.

Integration tests

- FCM
- Payment
- Database
- Queue

---

135. CONCURRENCY TEST

Explicitly test:

```
100 customers
same branch
same staff
same slot
```

Verify that only valid bookings succeed.

---

136. SECURITY TESTING

Test:

- SQL injection
- XSS
- CSRF
- IDOR
- Broken access control
- Authentication bypass
- Privilege escalation
- Token replay

- Duplicate payments
- Duplicate bookings
- Tenant leakage
- File upload abuse
- Rate-limit bypass

---

137. PRODUCTION ACCEPTANCE CRITERIA

The platform is not complete until:

Customer

Can discover a shop, select service/staff/time, pay online and receive confirmation.

Owner

Can register and configure their business.

Branch

Can be created and independently managed.

Staff

Can have service-specific schedules.

Booking

Cannot be duplicated through race conditions.

Payment

Must be backend verified.

Cancellation

Uses owner-configured time limit.

Notifications

Work through FCM and in-app notification center.

Marketing

Owners can create permitted offers/campaigns.

Discovery

Customers can find nearby businesses.

Sponsored

Sponsored businesses are clearly labelled.

Remote UI

Configuration/theme/content changes can happen without APK update.

App updates

New native versions can be distributed from official website.

Billing

₹1 billing exists but is disabled initially.

Billing activation

Super Admin can activate it.

Billing usage

One chargeable booking creates one usage record.

Suspension

Unpaid tenants can be suspended.

Recovery

Payment restores access.

Security

Tenant isolation is tested.

Performance

Concurrent booking is tested.

Backup

Database restoration is tested.

138. AGENT INPUT REQUEST SYSTEM — CRITICAL

The development agent, Antigravity, is allowed and expected to request missing external inputs from the user during development.

This ensures the build does not stop because credentials, configuration files or third-party setup information are missing.

Examples include:

- Firebase FCM JSON
- Firebase project information
- Payment gateway keys
- SMTP credentials
- Google Maps API key
- Domain information
- SSL information
- Webhook secrets
- Storage credentials
- CDN credentials
- OAuth credentials
- App signing keys
- External API keys

The agent must continue development while waiting for these values whenever possible.

---

139. AGENT INPUT REQUEST RULE

If an external input is missing:

1. Continue building everything that does not require it.
2. Create the proper service/interface/adapter.
3. Create configuration placeholders.
4. Mark the integration:

```
PENDING_USER_INPUT
```

1. Clearly tell the user what is required.

Never invent credentials.

Never use fake production credentials.

Never expose secrets in source code.

---

140. STANDARD INPUT REQUEST FORMAT

When an input is needed, show:

```
REQUIRED INPUT FROM USER

Field:
Purpose:
Where it is used:
Required format:
How to obtain it:
```

Example:

```
Field:
Firebase service-account JSON

Purpose:
FCM push notifications

Where used:
Laravel notification service

Required format:
JSON file

How to obtain:
Firebase Console → Project Settings → Service Accounts
```

---

141. SAFE PLACEHOLDER ARCHITECTURE

If credentials are unavailable:

```
FCMService
↓
MockFCMService
```

or:

```
PaymentService
↓
MockPaymentService
```

or:

```
MapsService
↓
MockLocationService
```

The application must remain buildable.

Do NOT pretend a mock service is production functionality.

---

142. EXTERNAL CONFIGURATION

Keep external configuration separated.

Backend configuration should use environment variables/config files.

Example:

```
config/
├── firebase.php
├── payment.php
├── maps.php
├── mail.php
└── storage.php
```

Flutter:

```
lib/core/config/
```

Do not place private server credentials inside Flutter.

---

143. ENVIRONMENT REQUEST FILES

Generate:

```
.env.example
.env.required
```

`.env.required` should clearly list missing configuration.

Example:

```
FCM_SERVICE_ACCOUNT
PAYMENT_PROVIDER_KEY
PAYMENT_PROVIDER_SECRET
MAPS_API_KEY
```

---

144. CONTINUOUS BUILD POLICY

Missing external credentials must NOT stop unrelated development.

Example:

If FCM credentials are missing:

Build:

- Notification database
- Notification API
- Notification UI
- Device registration
- FCM service interface
- Mock service

Then request the actual FCM credentials.

---

145. PRODUCTION CREDENTIAL RULE

Production is only considered ready after:

- Required credentials provided
- External services tested
- Webhooks verified
- Push notifications tested
- Payments tested
- APK signing configured
- Domain/SSL configured

---

146. AUTOMATIC DEPENDENCY INSTALLATION

Antigravity should inspect installed tooling.

If missing:

- Composer dependencies
- Flutter packages
- Dart packages
- Node packages if required

install only required dependencies.

Do not install unnecessary packages.

After installation:

```
flutter pub get
composer install
```

or equivalent appropriate commands.

Verify compatibility.

---

147. PACKAGE QUALITY RULE

Prefer:

- Stable

- Maintained
- Widely used
- Compatible
- Secure

packages.

Do not add dependencies simply because they are convenient.

Before adding a package, determine whether native/framework functionality already solves the requirement.

---

148. CODE QUALITY

Follow:

- SOLID principles where useful
- Separation of concerns
- DRY
- Clear naming
- Service classes
- DTOs where useful
- Form requests
- Policies
- Repositories only where justified
- Events
- Jobs
- Tests

Do not create unnecessary abstractions.

---

149. LARAVEL STRUCTURE

Recommended:

```
app/
├── Actions/
├── Console/
├── DTOs/
├── Enums/
├── Events/
├── Exceptions/
```

```
├── Http/
│   ├── Controllers/
│   ├── Middleware/
│   ├── Requests/
│   └── Resources/
├── Jobs/
├── Models/
├── Notifications/
├── Policies/
├── Services/
│   ├── Availability/
│   ├── Booking/
│   ├── Billing/
│   ├── Notification/
│   ├── Payment/
│   ├── Promotion/
│   └── Tenant/
└── Support/
```

Controllers should remain thin.

Business logic belongs in appropriate services/actions.

---

150. DATABASE MIGRATIONS

All schema changes must use migrations.

Never manually depend on an undocumented database state.

Create:

- Migrations
- Seeders
- Factories
- Test data

---

151. API DOCUMENTATION

Document:

- Authentication
- Endpoints

- Request format
- Response format
- Validation
- Errors
- Permissions
- Examples

OpenAPI/Swagger is recommended.

---

152. ERROR HANDLING

Flutter should show user-friendly messages.

Backend should return structured errors.

Logs should contain technical details.

Never expose:

- SQL
- Stack traces
- Secrets
- Internal paths

to users.

---

153. MAINTENANCE MODE

Super Admin should be able to enable:

```
MAINTENANCE MODE
```

Options:

- Entire platform
- Customer app
- Owner app
- Specific feature

Display a controlled maintenance screen.

Emergency administrative access should remain available to authorized administrators.

154. FEATURE ROLLOUT

Support gradual rollout.

Example:

```
New feature:
Waitlist

Enabled:
10% users

Then:
25%

Then:
100%
```

This is optional for MVP but architecture should support it.

---

155. APP CRASH RESILIENCE

The application should:

- Catch network errors
- Handle expired sessions
- Handle invalid remote config
- Handle malformed API responses
- Handle FCM token failure
- Handle payment interruption
- Recover from temporary database/cache issues

Never leave the app permanently stuck on a loading screen.

---

156. SESSION EXPIRATION

When access token expires:

```
Refresh
↓
If successful:
Continue

If failed:
Secure logout
↓
Login screen
```

Never expose tokens to logs.

---

157. PAYMENT INTERRUPTION

If customer closes app during payment:

When returning:

```
Check payment status from server
```

Do not create another booking automatically.

---

158. BOOKING RECOVERY

If payment succeeded but client failed before receiving confirmation:

Customer opens app:

```
Server checks pending booking/payment
```

and recovers the correct status.

This prevents:

```
Money paid
but booking appears lost
```

---

159. CUSTOMER NOTIFICATION RECOVERY

If FCM fails:

The notification must still exist in the in-app notification database.

When user opens the app:

```
Fetch unread notifications
```

---

160. OWNER NOTIFICATION RECOVERY

New booking notifications must be persisted.

Even if push delivery fails, the owner sees:

```
New Booking
```

inside the application.

---

161. DATA CONSISTENCY

The backend must maintain consistency between:

```
Booking
Payment
Slot
Staff
Billing
Notification
```

Use transactions where necessary.

---

162. SECURITY OF REMOTE CONTENT

Remote content must be validated.

Do not allow arbitrary scripts.

URLs must be validated.

Images must come from trusted/approved sources where appropriate.

---

163. MARKETING SAFETY

Promotional system must have:

- Consent
- Frequency limits
- Unsubscribe/disable options
- Campaign expiration
- Audit trail

Do not spam users.

---

164. CUSTOMER PRIVACY

Do not expose full customer lists to unauthorized staff.

Staff should only see customer information necessary to perform their job.

Owner can manage business customers according to platform rules.

---

165. OWNER DATA PRIVACY

Tenant financial data must never be exposed to:

- Other owners
- Staff without permission
- Customers
- Public APIs

---

166. SUPER ADMIN DATA ACCESS

Super Admin access should be logged.

Sensitive data access should be auditable.

---

167. FUTURE AI FEATURES

Architecture can later support:

- Demand prediction
- Best pricing suggestions
- Customer churn prediction
- Recommended campaigns
- Staff utilization insights
- Intelligent slot optimization

AI recommendations must remain recommendations unless explicitly approved.

---

168. FUTURE MARKETPLACE FEATURES

Eventually:

```
Compare salons
Compare services
Popular near you
Trending
Offers
Recommended
```

---

169. FUTURE FRANCHISE SUPPORT

Multi-branch architecture should allow future:

```
Franchise
↓
Multiple owners/managers
↓
Multiple locations
```

without breaking tenant architecture.

---

170. FUTURE WHITE-LABEL SUPPORT

The architecture should eventually support:

```
Business-specific branding
Custom domain
Custom app identity
```

---

171. FINAL PRODUCT EXPERIENCE

Customer experience:

```
Open App
↓
See nearby salons
↓
See what's open
↓
See offers
↓
Choose shop
↓
Choose service
↓
Choose staff
↓
Choose time
↓
Pay
↓
Receive confirmation
↓
Get reminders
↓
Complete service
↓
Review
↓
Receive rebooking/offer
```

```
↓
Book again
```

Owner experience:

```
Register
↓
Configure business
↓
Add branch
↓
Add services
↓
Add staff
↓
Configure schedule
↓
Enable online payment
↓
Receive bookings
↓
Manage customers
↓
Promote offers
↓
View analytics
↓
Increase utilization
↓
Grow revenue
```

SaaS owner experience:

```
Launch free
↓
Acquire businesses
↓
Acquire customers
↓
Increase booking volume
↓
Activate billing
↓
₹1 / chargeable booking
```

```
↓
Generate invoices
↓
Collect payments
↓
Manage platform growth
```

---

172. IMPLEMENTATION PHASES

PHASE 1 — FOUNDATION

Build:

- Laravel
- MySQL
- Authentication
- RBAC
- Tenant architecture
- REST API
- Flutter foundation
- Admin foundation

PHASE 2 — BUSINESS MANAGEMENT

Build:

- Owner registration
- Branches
- Services
- Combos
- Staff
- Staff-service mapping
- Working hours
- Breaks
- Holidays

PHASE 3 — BOOKING ENGINE

Build:

- Availability

- Pricing
- Dynamic pricing
- Staff selection
- Any staff
- Slot locking
- Booking
- Cancellation
- Rescheduling
- History

PHASE 4 — PAYMENTS

Build:

- Payment abstraction
- Gateway integration
- Payment order
- Webhooks
- Verification
- Recovery

PHASE 5 — NOTIFICATIONS

Build:

- FCM
- Device registration
- Notification center
- Booking notifications
- Reminders
- Campaign notifications

PHASE 6 — CUSTOMER EXPERIENCE

Build:

- Discovery
- Location
- Search
- Nearby
- Shop page
- Favourites

- Reviews
- History
- Rebooking

PHASE 7 — OWNER GROWTH

Build:

- CRM
- Segmentation
- Promotions
- Offers
- Last-minute slots
- Campaigns
- Customer notifications
- Analytics

PHASE 8 — ADMIN

Build:

- Super Admin
- Subadmin
- Roles
- Permissions
- Tenant management
- Announcements
- Advertisements
- Sponsored listings

PHASE 9 — REMOTE PLATFORM

Build:

- Remote theme
- Remote configuration
- Feature flags
- Server-controlled home
- Dynamic banners
- App version system
- APK download page

PHASE 10 — SAAS BILLING

Build:

- Billing engine
- Usage ledger
- Monthly invoices
- ₹1 booking fee
- Payment
- Overdue
- Suspension
- Restoration

Keep billing OFF.

PHASE 11 — OPTIMIZATION

Build:

- Redis
- Cache
- Queues
- Scheduler
- Cleanup
- Database optimization
- Image optimization
- API optimization

PHASE 12 — SECURITY

Perform:

- Authentication testing
- Authorization testing
- Tenant isolation testing
- IDOR testing
- Payment testing
- Concurrency testing
- File upload testing
- Rate-limit testing

PHASE 13 — PRODUCTION

Prepare:

- Production environment
- SSL
- Database
- Queue worker
- Scheduler
- Backups
- Monitoring
- Release APK
- Official download page
- Documentation

---

173. FINAL DEFINITION OF DONE

The project is considered complete only when:

1. Laravel backend works.
2. MySQL schema works.
3. Authentication works.
4. RBAC works.
5. Tenant isolation works.
6. Owner registration works.
7. Branch management works.
8. Service management works.
9. Combo management works.
10. Staff management works.
11. Staff-service mapping works.
12. Working hours work.
13. Holiday/leave system works.
14. Dynamic pricing works.
15. Availability engine works.
16. Slot locking works.
17. Concurrent booking protection works.
18. Online payment works once credentials are supplied.
19. Payment verification works.
20. Cancellation works.
21. Rescheduling works.
22. Customer history works.
23. Customer discovery works.
24. Nearby search works.
25. Staff selection works.
26. FCM works once credentials are supplied.

27. In-app notifications work.
28. Marketing campaigns work.
29. Notification frequency limits work.
30. CRM works.
31. Promotions work.
32. Sponsored placement works.
33. Owner analytics work.
34. Multi-branch analytics work.
35. Super Admin works.
36. Subadmin permissions work.
37. Remote configuration works.
38. Remote theme works.
39. Feature flags work.
40. APK update checking works.
41. Official APK download works.
42. Billing engine exists.
43. Billing remains disabled initially.
44. ₹1 can be activated from Super Admin.
45. Usage ledger is idempotent.
46. Monthly invoices work.
47. Suspension works.
48. Payment restoration works.
49. Automated cleanup works.
50. Backup works.
51. Security tests pass.
52. Concurrency tests pass.
53. Production build succeeds.
54. Documentation is complete.

---

174. FINAL ANTIGRAVITY COMMAND

Build this as a real commercial SaaS.

Do not build a demo.

Do not build only CRUD screens.

Do not stop after generating UI.

Do not fake payment success.

Do not fake FCM.

Do not trust frontend authorization.

Do not allow tenant data leakage.

Do not duplicate business logic between applications.

Do not hard-code ₹1.

Do not require APK installation for changes that can safely be handled through remote configuration.

Do not download executable code remotely.

Do not spam users.

Do not expose customer data unnecessarily.

Do not delete important records during cleanup.

Do not stop development because an external credential is missing.

Instead:

```
BUILD
↓
TEST
↓
FIX
↓
VERIFY
↓
REQUEST REQUIRED INPUT
↓
CONTINUE
```

The application must initially operate as a completely free SaaS.

The SaaS owner must later be able to activate:

```
₹1 per chargeable booking
```

without rebuilding the entire platform.

The long-term goal is to create one of the most complete appointment and business-growth platforms for salons and barbers, combining:

```
BOOKING
+
PAYMENT
+
STAFF
+
BRANCHES
+
CUSTOMERS
+
CRM
+
MARKETING
+
PROMOTIONS
+
DISCOVERY
+
SPONSORED LISTINGS
+
ANALYTICS
+
NOTIFICATIONS
+
BUSINESS GROWTH
+
SAAS BILLING
```

The backend must remain the authoritative source of truth for:

```
AUTHENTICATION
AUTHORIZATION
TENANT ISOLATION
AVAILABILITY
BOOKING
PAYMENT
BILLING
```

The first implementation target is Android + customer web + admin web, with the backend designed from day one so iOS can be added later.

Start by inspecting the supplied workspace.

Then create the architecture.

Then implement the project phase by phase.

Run the application after each major phase.

Run tests continuously.

Fix errors instead of working around them.

Do not claim completion until the feature has been verified end-to-end.

END OF MASTER PRD